Felhők hálózati szolgáltatásai laboratórium

BMEVITMMB11

OCR alapú felhőalkalmazás

Fejlesztői dokumentáció

Horváth Benedek

D86EP7

Budapest, 2026

Tartalomjegyzék

1. Bevezetés ... 3
2. CI/CD környezet ... 3
   2.1. Architektúra áttekintés ... 3
   2.2. Futási környezet ... 3
   2.3. K3s telepítése ... 3
   2.4. ArgoCD telepítése ... 4
   2.5. GitHub repó és projekt struktúra ... 4
   2.6. Dockerfile ... 5
   2.7. Helm templates ... 6
   2.8. GitHub Actions CI pipeline ... 7
   2.9. GitHub Secrets ... 10
   2.10. ArgoCD alkalmazás ... 10
   2.11. Publikus elérés ... 10
   2.12. Működés ellenőrzése ... 11
3. Weboldal és OCR detektálás ... 12
   3.1. Architektúra ... 12
   3.2. Stack ... 12
   3.3. Projektstruktúra ... 12
   3.4. Adatszerkezet ... 13
   3.5. Web konténer (app/web) ... 13
   3.6. Worker konténer (app/worker) ... 14
   3.7. Kafka ... 14
   3.8. Helm templates ... 14
   3.9. GitHub Actions CI pipeline ... 15
   3.10. Működés ellenőrzése ... 16

1. Bevezetés

A házi feladat célja egy OCR (optikai karakterfelismerés) alapú webszolgáltatás megvalósítása felhős ökoszisztémában. Jelen dokumentáció első verziója a kialakított CI/CD környezetet ismerteti.

2. CI/CD környezet

2.1. Architektúra áttekintés

A CI/CD pipeline a következő folyamatot valósítja meg:

1. A fejlesztő push-ol a `master` branch-re
2. A GitHub Actions workflow buildeli a Docker image-et
3. Az image feltöltésre kerül a Docker Hub-ra
4. A workflow frissíti a Helm `values.yaml` fájlt az új image tag-gel (commit SHA)
5. Az ArgoCD észleli a változást és automatikusan deployolja az új verziót a K3s klaszterbe

2.2. Futási környezet

A fejlesztés és futtatás egy otthoni Fedora szerveren történik, SSH kapcsolaton keresztül.

OS	Fedora Server
Kubernetes	K3s Fedora miatt
CD	ArgoCD
CI	GitHub Actions
Container registry	Docker Hub
Package	Helm

2.3. K3s telepítése

A laboroktól eltérően minikube helyett K3s-t választottam. A K3s egy lightweight Kubernetes disztribúció, amelyet edge és szerver környezetekbe terveztek. Natívan fut Fedoran, nem igényel VM-et vagy Docker-in-Docker megoldást, és alacsony erőforrásigényű.

```
curl -sfL https://get.k3s.io | sh -
```

1. kódrészlet: K3s telepítése

```
bede in 🌐 bedelab in cloud-lab-hw on master
❯ kubectl get nodes
NAME       STATUS   ROLES           AGE   VERSION
bedelab    Ready    control-plane   27m   v1.34.5+k3s1
```

1. ábra: K3s klaszter állapota

2.4. ArgoCD telepítése

A laborról ismerős ArgoCD-t választottam. Az ArgoCD olyan CD eszköz, amely a Git repóban tárolt manifesteket automatikusan szinkronizálja a klaszterrel. A telepítés így nézett ki:

```
kubectl create namespace argocd
kubectl apply -n argocd -f \
  https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/
install.yaml
kubectl patch svc argocd-server -n argocd \
  -p '{"spec": {"type": "NodePort"}}'
```

2. kódrészlet: ArgoCD telepítése a K3s klaszterre

```
bede in 🌐 bedelab in cloud-lab-hw on master
❯ kubectl -n argocd get pods
NAME                                                READY   STATUS             RESTARTS        AGE
argocd-application-controller-0                     1/1     Running            0               24m
argocd-applicationset-controller-7dc85c9598-9j8mr   0/1     CrashLoopBackOff   6 (4m49s ago)   24m
argocd-dex-server-7d88b5b798-m4nn2                  1/1     Running            0               24m
argocd-notifications-controller-5f95df7f8c-vvm7l    1/1     Running            0               24m
argocd-redis-65b5bbd8cb-vztk2                       1/1     Running            0               24m
argocd-repo-server-69fb9fb66f-w7zb6                 1/1     Running            0               24m
argocd-server-58dbdffdcd-5zrm8                      1/1     Running            0               24m
```

2. ábra: ArgoCD podok állapota

2.5. GitHub repó és projekt struktúra

A repositorí felépítése megegyezik a laborokéval. Az app almappa tartalmazza az alkalmazást és a docker imaget felépítő Dockerfilet. A helm mappa pedig az erőforrások létrehozásáért felel.

```
bede in 🌐 bedelab in cloud-lab-hw on ᚠ master
❯ tree ~/projects/cloud-lab-hw
/home/bede/projects/cloud-lab-hw
├── app
│   ├── app.py
│   └── Dockerfile
└── helm
    ├── Chart.yaml
    ├── templates
    │   ├── deployment.yaml
    │   └── service.yaml
    └── values.yaml

4 directories, 6 files
```

3. ábra: Repó struktúra

2.6. Dockerfile

A laborról ismerős Flask alkalmazást használom placeholder-ként, amely majd az OCR funkciókat valósítja meg.

```python
from flask import Flask

app = Flask(__name__)

@app.route("/health")
def health():
    return "OK"

@app.route("/")
def index():
    return "Hello Cloud Lab OCR App!"

if __name__ == '__main__':
    app.run(host="0.0.0.0", port=5000)
```

3. kódrészlet: Flask alkalmazás (app/app.py)

A Dockerfile alapján épül fel a Python appot tartalmazó minimális konténer.

```dockerfile
FROM python:3.12-slim
RUN pip install flask
WORKDIR /app
COPY app.py .
CMD ["python", "-u", "app.py"]
```

4. kódrészlet: Dockerfile (app/Dockerfile)

2.7. Helm templates

A Kubernetes erőforrások leírásához Helm chart-ot készítettem. A `deployment.yaml` definiálja a futtatandó pod-ot, megadja az image nevét és verzióját (a `values.yaml`-ből), a konténer portját, valamint readiness és liveness probe-okat a `/health` végponton, ezek biztosítják, hogy a Kubernetes automatikusan újraindítsa a pod-ot, ha az alkalmazás nem válaszol.

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ocr-app
spec:
  replicas: 1
  selector:
    matchLabels:
      app: ocr-app
  template:
    metadata:
      labels:
        app: ocr-app
    spec:
      containers:
        - name: ocr-app
          image: benhorv/ocr-app:{{ .Values.env.APP_VERSION }}
          ports:
            - name: http
              containerPort: 5000
          readinessProbe:
            httpGet:
              path: /health
              port: 5000
          livenessProbe:
            httpGet:
              path: /health
              port: 5000
```

5. kódrészlet: Kubernetes Deployment (helm/templates/deployment.yaml)

A `service.yaml` hálózati végpontot biztosít a pod számára. Egyszerűség kedvéért NodePort típusú, így kívülről is látszódik, és ki tudom rakni Cloudflare Tunnellel (erről később lesz szó). Ez a megoldás most egyszerűbb a K3s miatt is.

```yaml
apiVersion: v1
kind: Service
metadata:
  name: ocr-app
spec:
  type: NodePort
  ports:
    - port: 5000
      targetPort: http
      protocol: TCP
      name: http
  selector:
    app: ocr-app
```

6. kódrészlet: Kubernetes Service (helm/templates/service.yaml)

A Helm `values.yaml` fájlban az `APP_VERSION` értéket a GitHub Actions workflow minden push után automatikusan frissíti a commit SHA-ra, így az ArgoCD mindig a legfrissebb image-et deployolja.

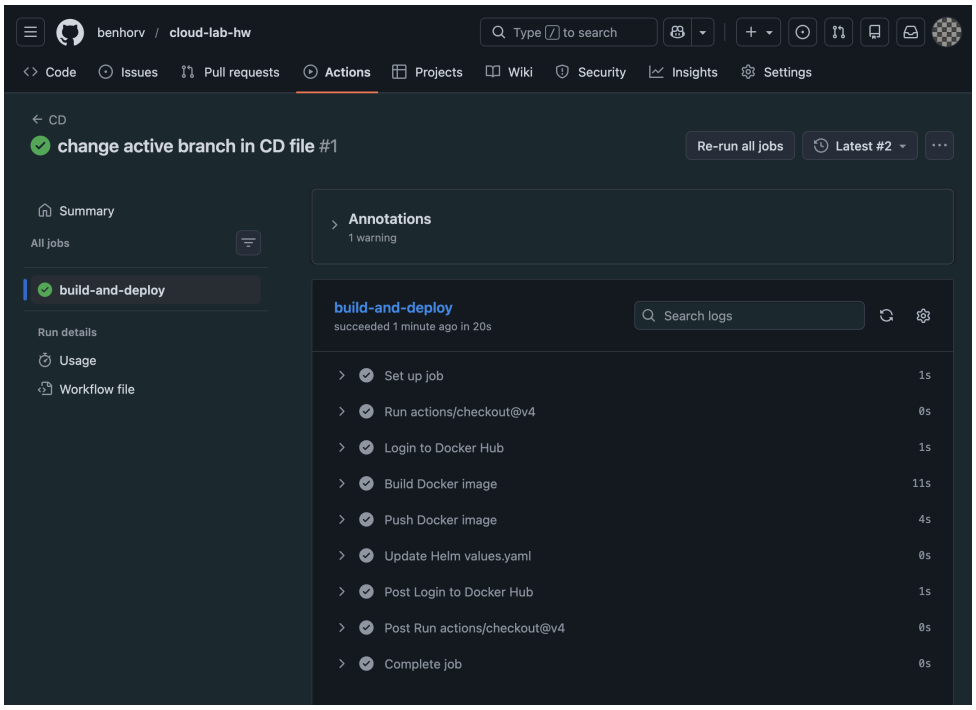
2.8. GitHub Actions CI pipeline

A `master` branch-re történő push hatására a workflow:

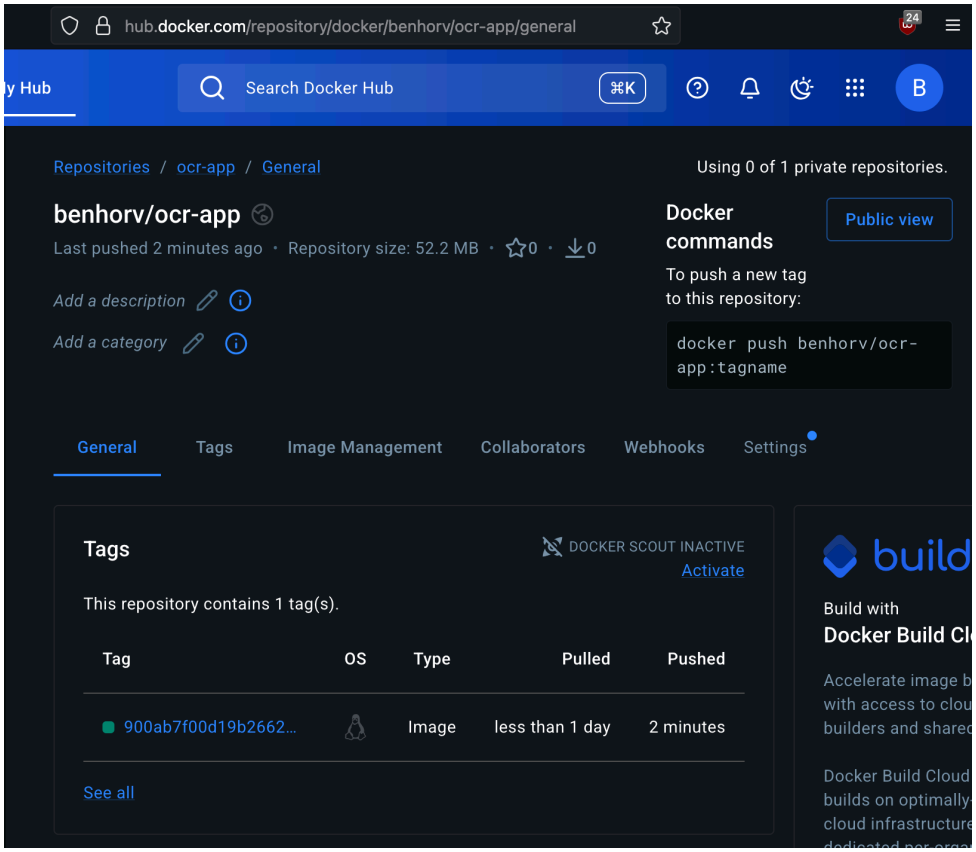
- Buildeli a Docker image-et (commit SHA tag-gel)
- Push-olja a Docker Hub-ra
- Frissíti a Helm `values.yaml`-t az új tag-gel

```yaml
name: CD
on:
  push:
    branches: [master]
env:
  DOCKERHUB_USERNAME: ${{ secrets.DOCKER_USERNAME }}
  DOCKERHUB_KEY: ${{ secrets.DOCKER_KEY }}
  IMAGE_NAME: ocr-app
jobs:
  build-and-deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Login to Docker Hub
        uses: docker/login-action@v3
        with:
          username: ${{ env.DOCKERHUB_USERNAME }}
          password: ${{ env.DOCKERHUB_KEY }}
      - name: Build Docker image
        run: cd app && docker build -t
          ${{ env.DOCKERHUB_USERNAME }}/${{ env.IMAGE_NAME }}:
${{ github.sha }} .
      - name: Push Docker image
        run: docker push
          ${{ env.DOCKERHUB_USERNAME }}/${{ env.IMAGE_NAME }}:
${{ github.sha }}
      - name: Update Helm values.yaml
        run: |
          cd helm
          sed -i 's|APP_VERSION:.*|APP_VERSION: '${{ github.sha }}'|'
values.yaml
          git config --global user.name 'GitHub Actions'
          git config --global user.email 'actions@github.com'
          git add values.yaml
          git commit -m "Update image tag to ${{ github.sha }}"
          git push
```

7. kódrészlet: GitHub Actions workflow (cd.yml)



4. ábra: GitHub Actions sikeres futás



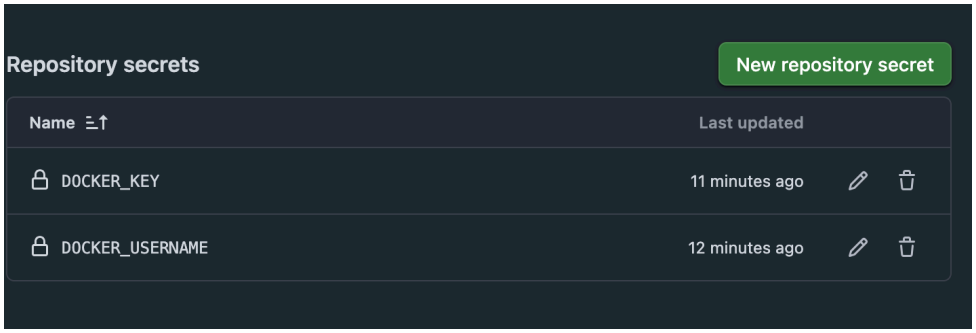
5. ábra: Docker Hub image

Fent látható, hogy a CD fájl alapján a megfelelő folyamatok lefutottak, lefutott az action GitHubon, és feltöltődött az image DockerHubra, amit majd az ArgoCD húz le.

2.9. GitHub Secrets

A Docker Hub credentials titkosított GitHub Secrets-ként vannak tárolva:

- `DOCKER_USERNAME`: Docker Hub felhasználónév
- `DOCKER_KEY`: Docker Hub access token

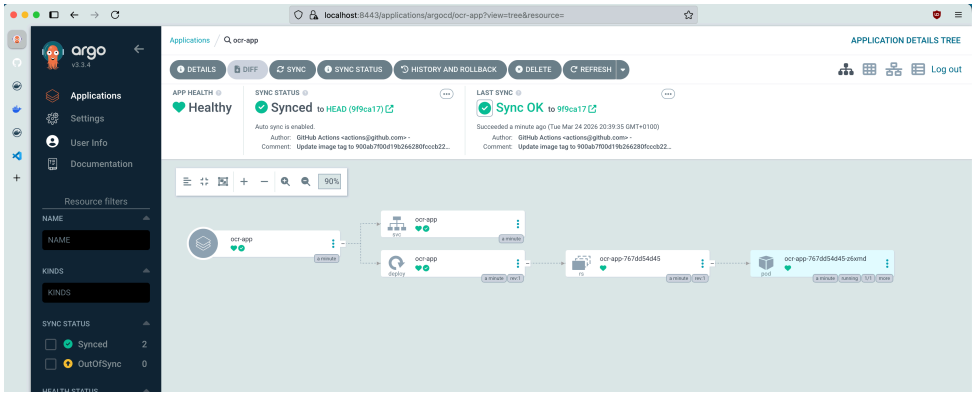


6. ábra: GitHub Secrets beállítás

2.10. ArgoCD alkalmazás

Az ArgoCD a következő beállításokkal figyeli a repót:

- **Sync Policy**: Automatic
- **Prune Resources**: enable
- **Self Heal**: enable



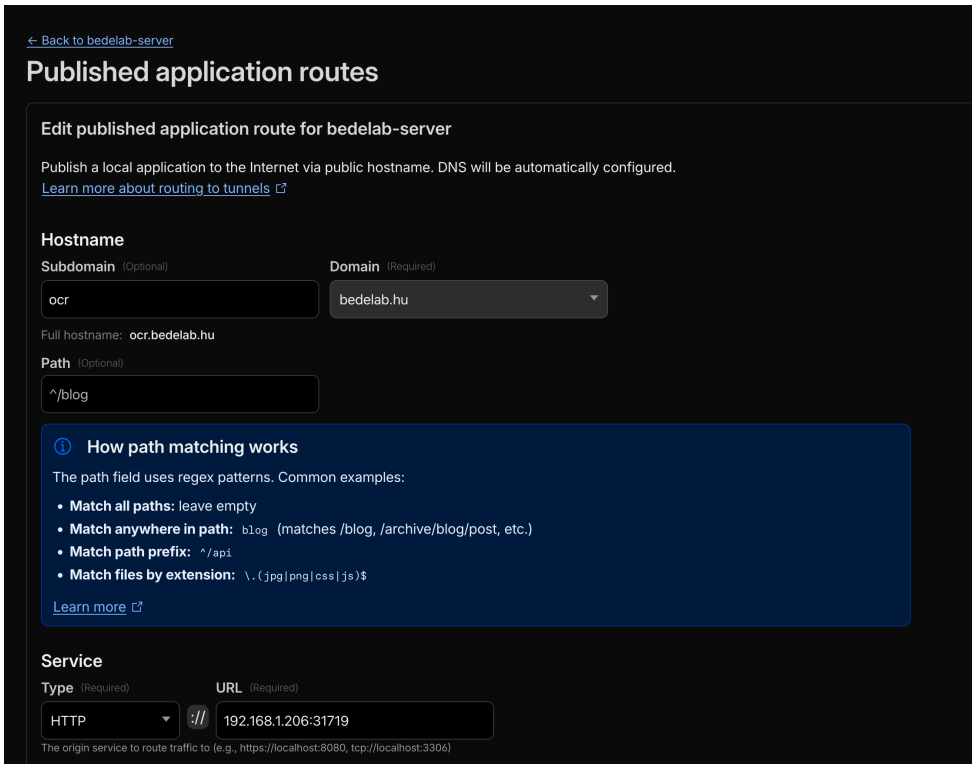
7. ábra: ArgoCD alkalmazás állapota

Látszik, hogy az ArgoCD megfelelő beállítás után sikeresen lehúzta a megfelelő imaget, és elindult az alkalmazás.

2.11. Publikus elérés

Nem a házi feladat szerves része, de a környezetből adódóan kihasználom, hogy már futnak szolgáltatások a szerveremen egy publikus DNS címen. Ezek a szolgálatások Cloudflare Tunnel-en keresztül érhetők el, így ez már be volt állítva, egyszerűen hozzá tudtam adni az OCR appot. A tunnel a K3s NodePort szolgáltatásra (`localhost:31719`) irányít, így az alkalmazás a `https://ocr.bedelab.hu` címen érhető el külső hálózatról is, anélkül hogy a szerveren portot

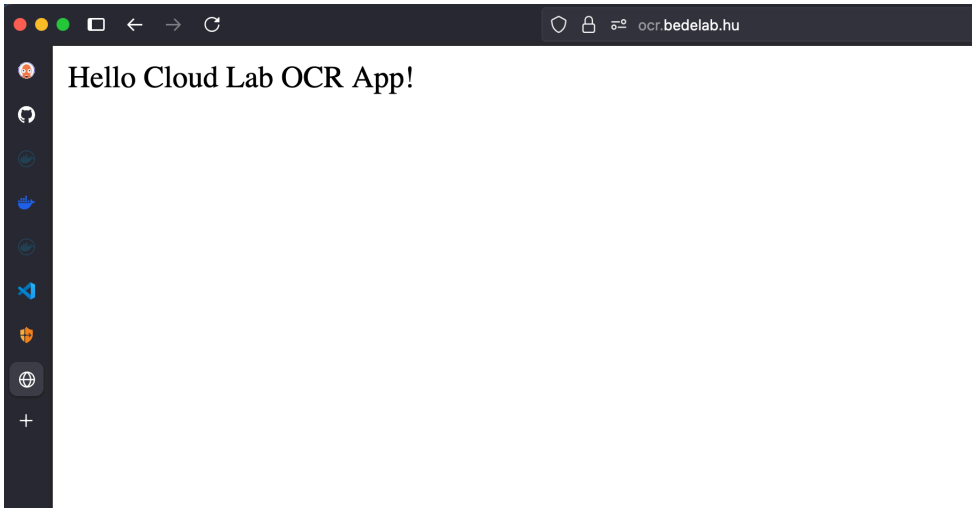
kellene nyitni a tűzfalon. Így biztonságosan, HTTPS-en keresztül elérhető, anélkül, hogy SSH-val kéne ügyeskednem, és ellenőrizhető is a feladat. :)



8. ábra: Cloudflare Tunnel konfiguráció

2.12. Működés ellenőrzése

Az alkalmazás a K3s klaszterben fut, NodePort-on érhető el lokálisan, és Cloudflare Tunnel-en keresztül publikusan a `https://ocr.bedelab.hu` címen.



9. ábra: Az alkalmazás a böngészőben

3. Weboldal és OCR detektálás

3.1. Architektúra

A második részfeladatban a placeholder Flask alkalmazást egy valódi OCR webszolgáltatással váltottam fel. A rendszer két külön konténerből áll, amelyek egy Kafka üzenetsoron keresztül kommunikálnak egymással. Ez a megközelítés megfelel a felhős mikroszolgáltatás-elveknek, a webapp és az OCR egymástól függetlenül skálázható és futtatható.

Így néz ki egy szekvencia:

1. A user feltölti a képet
2. A `web` konténer elmenti a képet a megosztott PVC-re (`/data/<uuid>/`), majd üzenetet küld a Kafka `ocr-jobs` topicba
3. A `worker` konténer megkapja az üzenetet, lefuttatja az OCR-t, és az eredményt beírja a `meta.json` fájlba
4. A user a képre kattintva láthatja az annotált verziót, amin a detektált szavak jelölve vannak

3.2. Stack

OCR	Tesseract (pytesseract)
Képfeldolgozás	Pillow
Üzenetsor	Kafka
Tárolás	Kubernetes PVC (K3s lokális tárhely)
Webszerver	Flask

A Tesseract mellett döntöttem, mivel konténerben futtatható, nem igényel külső API-t. A Kafka a laboron megismert megoldás, ezért választottam. Lehet kicsit előre dolgoztam a Kafkával, de a 3. feladatrészhez jól jön, hogy már megvan.

3.3. Projektstruktúra

Az `app` mappa két alkönyvtárra bővült:

```
app/
  web/          # Flask webalkalmazás
    app.py
    Dockerfile
    templates/
      index.html
  worker/       # OCR feldolgozó
    worker.py
    Dockerfile
```

8. kódrészlet: Frissített projekt struktúra

3.4. Adatszerkezet

Minden feltöltött kép egy saját könyvtárban tárolódik a PVC-n:

```
/data/
  <uuid>/
    image<ext>    // az eredeti feltöltött kép
    meta.json     // leírás + OCR eredmények
```

9. kódrészlet: Tárolási struktúra a PVC-n

A `meta.json` felépítése:

```
{
  "description": "Példa leírás",
  "text": "Detektált szöveg egy sorban",
  "ext": ".png",
  "status": "done",
  "words": [
    {"text": "Szó", "x": 10, "y": 20, "w": 50, "h": 15}
  ]
}
```

10. kódrészlet: meta.json adatszerkezet

3.5. Web konténer (app/web)

A Flask alkalmazás három végpontot valósít meg:

- `GET /`: korábbi feltöltések listája leírással és detektált szöveggel
- `POST /upload`: kép mentése a PVC-re, üzenet küldése a Kafka `ocr-jobs` topicba, átirányítás
- `GET /image/<id>`: annotált kép megjelenítése/kiszolgálása, ezzel dolgozik a Pillow is

```
FROM python:3.12-slim
RUN pip install flask pillow kafka-python
WORKDIR /app
COPY app.py .
COPY templates/ templates/
CMD ["python", "-u", "app.py"]
```

11. kódrészlet: Web Dockerfile (app/web/Dockerfile)

3.6. Worker konténer (app/worker)

A worker egy Kafka consumer, amely folyamatosan figyeli az `ocr-jobs` topicot. Minden üzenet érkezésekor beolvassa a képet a PVC-ről, lefuttatja a Tesseract OCR-t, majd visszaírja az eredményt a `meta.json` fájlba.

```
FROM python:3.12-slim
RUN apt-get update && apt-get install -y tesseract-ocr \
    && rm -rf /var/lib/apt/lists/*
RUN pip install pytesseract pillow kafka-python
WORKDIR /app
COPY worker.py .
CMD ["python", "-u", "worker.py"]
```

12. kódrészlet: Worker Dockerfile (app/worker/Dockerfile)

3.7. Kafka

A Kafka a `default` névtérben fut egy egyszerű Deployment-ként, a laboron megismert `bashj79/kafka-kraft` image alapján. A `KAFKA_ADVERTISED_LISTENERS` környezeti változóval a K8s service hostname-re van konfigurálva, hogy más névterekből is elérhető legyen.

```
env:
  - name: KAFKA_ADVERTISED_LISTENERS
    value: "PLAINTEXT://kafka.default.svc.cluster.local:9092"
```

13. kódrészlet: Kafka advertised listener beállítása

3.8. Helm templates

A `deployment.yaml` két deploymentet definiál (`ocr-web` és `ocr-worker`), mindkettő ugyanazt a PVC-t mountolja `/data` alá. A `values.yaml` külön verziótageket tárol a két image-hez, amelyeket a CI pipeline frissít.

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: ocr-data
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
```

14. kódrészlet: PersistentVolumeClaim (helm/templates/pvc.yaml)

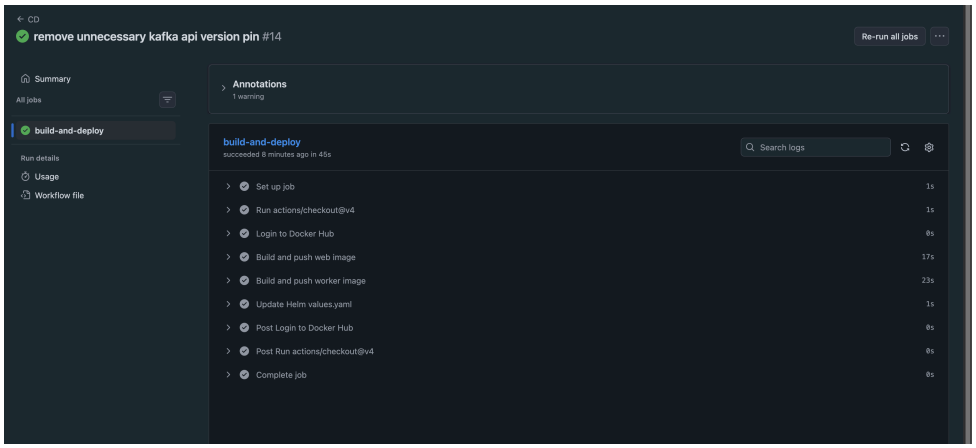
3.9. GitHub Actions CI pipeline

A CI pipeline kiegészült a két image buildelésével és pusholásával, valamint a `values.yaml` mindkét verziótagjének frissítésével.

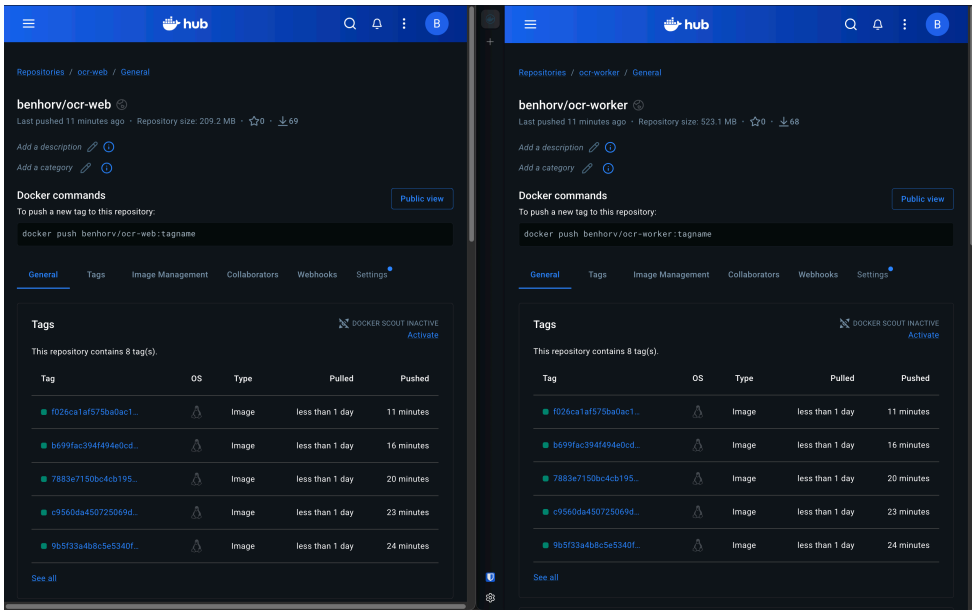
```
- name: Build and push web image
  run: |
    docker build -t ${{ env.DOCKERHUB_USERNAME }}/ocr-web:${{ github.sha }}
app/web
    docker push ${{ env.DOCKERHUB_USERNAME }}/ocr-web:${{ github.sha }}

- name: Build and push worker image
  run: |
    docker build -t ${{ env.DOCKERHUB_USERNAME }}/ocr-worker:
${{ github.sha }} app/worker
    docker push ${{ env.DOCKERHUB_USERNAME }}/ocr-worker:${{ github.sha }}
```

15. kódrészlet: Frissített CI pipeline (cd.yml)



10. ábra: GitHub Actions-ben web és worker image buildelése



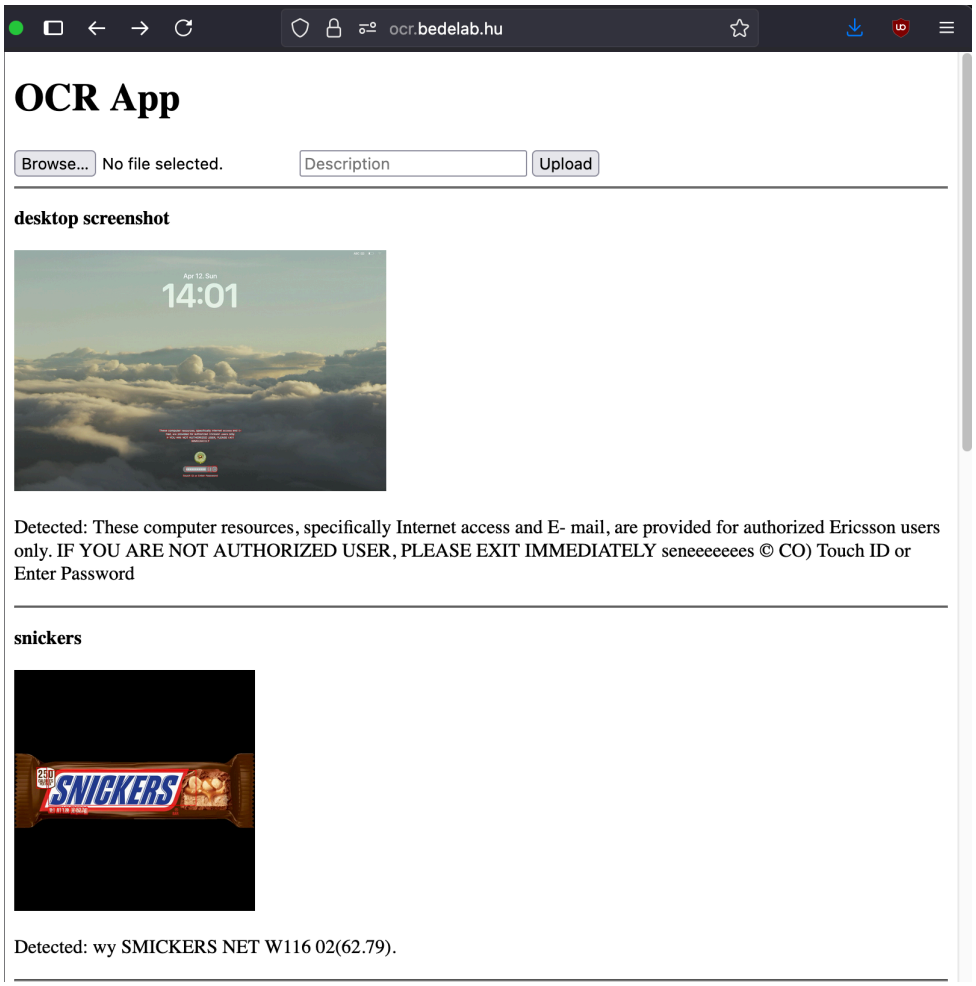
11. ábra: Docker Hub ocr-web és ocr-worker image-ek

3.10. Működés ellenőrzése

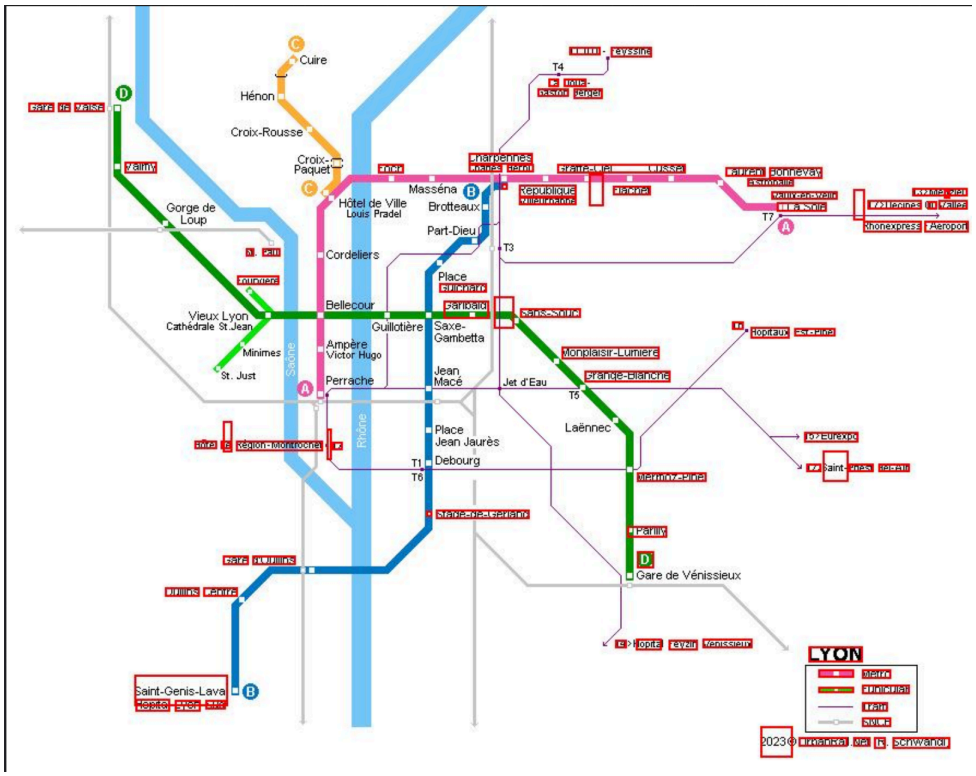
Az alkalmazás podjai a `kubectl get pods -n ocr-app` paranccsal ellenőrizhetők.

```
bede in 🌐 bedelab in ~
❯ kubectl get pods -n ocr-app
NAME                          READY   STATUS    RESTARTS   AGE
ocr-web-788594b979-svvp8      1/1     Running   0          11m
ocr-worker-585fb56dc-wpvzg    1/1     Running   0          11m
```

12. ábra: Futó podok állapota



13. ábra: A weboldal feltöltési formmal és OCR eredménnyel



14. ábra: Annotált kép a detektált szövegekkel